

Roteiro: juncoes e agregacoes com SQL Server

Base: curso_integridade_fiscal **Pre-requisitos:** SELECT, INNER JOIN, LEFT JOIN, WHERE e GROUP BY **Objetivo:** praticar agregacoes simples, diferenciar **WHERE** de **HAVING** e introduzir subconsultas sem fan-out.

Este roteiro usa efd_c100 como tabela de documentos da EFD. A palavra "EFD" neste material se refere a essa tabela.

Preparacao

1. Execute os scripts 01_criar_banco.sql a 05_inserir_fiscalizacao.sql.
2. Execute 06_criar_views.sql para criar as views de apoio.
3. Selecione o banco:

```
USE curso_integridade_fiscal;
GO
```

4. Confira a carga basica:

```
SELECT 'contribuinte' AS tabela, COUNT(*) AS quantidade FROM dbo.contribuinte
UNION ALL SELECT 'auditor_fiscal', COUNT(*) FROM dbo.auditor_fiscal
UNION ALL SELECT 'nfe', COUNT(*) FROM dbo.nfe
UNION ALL SELECT 'efd_c100', COUNT(*) FROM dbo.efd_c100
UNION ALL SELECT 'ordem_servico', COUNT(*) FROM dbo.ordem_servico
UNION ALL SELECT 'auto_infracao', COUNT(*) FROM dbo.auto_infracao;
```

Tabelas e chaves usadas

Tabela	Papel	Chave usada no roteiro
auditor_fiscal	cadastro do fiscal	matricula
ordem_servico	trabalho atribuido ao fiscal	matricula_auditor, id_os

Tabela	Papel	Chave usada no roteiro
auto_infracao	auto gerado durante uma OS	id_os, valor_principal
contribuinte	cadastro do emitente/declarante	id_contribuinte
nfe	documento fiscal emitido	id_emitente, chave_acesso
efd_c100	documento escriturado na EFD	id_contribuinte, chave_acesso

Questao 1 - Visao de produtividade por auditor

Objetivo

Criar uma visao que consolide cada auditor, suas ordens de servico e seus autos de infracao. Depois, comparar:

- **WHERE**: filtra linhas antes da agregacao;
- **HAVING**: filtra grupos depois da agregacao.

A atividade deve preservar os 20 auditores, inclusive aqueles sem OS ou sem auto.

Tabelas selecionadas

- `dbo.auditor_fiscal AS af`
- `dbo.ordem_servico AS os`
- `dbo.auto_infracao AS ai`

Passo 1 - Desenhar as juncoes

Escreva primeiro somente o **SELECT** das colunas de identificacao e verifique as juncoes:

```
SELECT af.matricula, af.nome, os.id_os, ai.id_auto,
       ai.valor_principal
```

```
FROM dbo.auditor_fiscal AS af
LEFT JOIN dbo.ordem_servico AS os
    ON os.matricula_auditor = af.matricula
LEFT JOIN dbo.auto_infracao AS ai
    ON ai.id_os = os.id_os;
```

Explique por que as duas juncoes sao **LEFT JOIN**. Troque temporariamente a primeira por **INNER JOIN** e observe se o numero de auditores muda.

Passo 2 - Criar a visao agregada

Crie a visao abaixo, preenchendo as funcoes de agregacao e os aliases indicados. A visao deve ter uma linha por auditor.

```
CREATE OR ALTER VIEW dbo.vw_roteiro_produtividade_auditor AS
SELECT af.matricula,
       af.nome,
       af.cargo,
       af.regiao_fiscal,
       COUNT(DISTINCT os.id_os) AS qtd_os,
       COUNT(DISTINCT CASE WHEN os.situacao = 'CONCLUIDA'
                           THEN os.id_os END) AS qtd_os_concluidas,
       COUNT(DISTINCT ai.id_auto) AS qtd_autos,
       COALESCE(SUM(ai.valor_principal), 0) AS valor_principal_total
FROM dbo.auditor_fiscal AS af
LEFT JOIN dbo.ordem_servico AS os
    ON os.matricula_auditor = af.matricula
LEFT JOIN dbo.auto_infracao AS ai
    ON ai.id_os = os.id_os
GROUP BY af.matricula, af.nome, af.cargo, af.regiao_fiscal;
GO
```

Passo 3 - Usar **WHERE** sobre a visao

A visao ja esta agregada. Portanto, neste SELECT o **WHERE** filtra linhas prontas, isto e, auditores consolidados:

```
SELECT matricula, nome, qtd_os, qtd_autos, valor_principal_total
FROM dbo.vw_roteiro_produtividade_auditor
WHERE valor_principal_total > 100000
ORDER BY valor_principal_total DESC;
```

Anote quantos auditores foram encontrados.

Passo 4 - Reproduzir o mesmo resultado com HAVING

Agora escreva a consulta diretamente sobre as tabelas. O filtro do valor total deve ficar no **HAVING**:

```
SELECT af.matricula, af.nome,
       COUNT(DISTINCT os.id_os) AS qtd_os,
       COUNT(DISTINCT ai.id_auto) AS qtd_autos,
       COALESCE(SUM(ai.valor_principal), 0) AS valor_principal_total
FROM dbo.auditor_fiscal AS af
LEFT JOIN dbo.ordem_servico AS os
      ON os.matricula_auditor = af.matricula
LEFT JOIN dbo.auto_infracao AS ai
      ON ai.id_os = os.id_os
GROUP BY af.matricula, af.nome
HAVING SUM(ai.valor_principal) > 100000
ORDER BY valor_principal_total DESC;
```

Compare as duas consultas. A primeira usa **WHERE** porque o valor já é uma coluna materializada da visao. A segunda usa **HAVING** porque o valor ainda está sendo calculado para cada grupo.

Passo 5 - Experimento com filtro de linha

Compare as consultas abaixo e explique por que elas não respondem a mesma pergunta:

```
-- Filtra OS antes de contar e somar
SELECT af.matricula, af.nome,
       COUNT(DISTINCT os.id_os) AS qtd_os,
       SUM(ai.valor_principal) AS valor_principal_total
FROM dbo.auditor_fiscal AS af
LEFT JOIN dbo.ordem_servico AS os
      ON os.matricula_auditor = af.matricula
LEFT JOIN dbo.auto_infracao AS ai
      ON ai.id_os = os.id_os
WHERE os.situacao = 'CONCLUIDA'
GROUP BY af.matricula, af.nome;

-- Agrega todas as OS e seleciona apenas grupos com OS concluida
SELECT af.matricula, af.nome,
       COUNT(DISTINCT os.id_os) AS qtd_os,
       SUM(ai.valor_principal) AS valor_principal_total
FROM dbo.auditor_fiscal AS af
LEFT JOIN dbo.ordem_servico AS os
```

```

ON os.matricula_auditor = af.matricula
LEFT JOIN dbo.auto_infracao AS ai
ON ai.id_os = os.id_os
GROUP BY af.matricula, af.nome
HAVING COUNT(DISTINCT CASE WHEN os.situacao = 'CONCLUIDA'
THEN os.id_os END) > 0;

```

Conferencia da questao 1

Verificacao	Resposta correta
Auditores cadastrados	20
Ordens de servico	150
Autos de infracao	72
Linhas da visao	20
Soma de qtd_os na visao	150
Soma de qtd_autos na visao	72
Consulta com WHERE valor_principal_total > 100000	executar e registrar
Consulta com HAVING SUM(valor_principal) > 100000	igual ao item anterior

Conferencia automatica:

```

SELECT COUNT(*) AS linhas,
SUM(qtd_os) AS total_os,
SUM(qtd_autos) AS total_autos
FROM dbo.vw_roteiro_produtividade_auditor;

```

Questao 2 - Contribuinte, NF-e e EFD

Objetivo

Consolidar, por contribuinte, a quantidade e o valor das NF-e de saida autorizadas e dos documentos regulares de saida escriturados na EFD. Usar **WHERE** para definir quais documentos entram na conta e **HAVING** para selecionar grupos com movimento relevante.

Tabelas selecionadas

- `dbo.contribuinte AS c`
- `dbo.nfe AS n`
- `dbo.efd_c100 AS e`

Passo 1 - Criar uma view para as NF-e

Nao junte `nfe` e `efd_c100` antes de agregar: um contribuinte pode ter muitos documentos nos dois lados. Primeiro crie uma view com uma linha por contribuinte e filtre os documentos no `WHERE` antes da agregacao:

```
CREATE OR ALTER VIEW dbo.vw_roteiro_nfe_contribuinte AS
SELECT n.id_emitente AS id_contribuinte,
       COUNT(*) AS qtd_nfe,
       SUM(n.valor_total) AS valor_nfe,
       SUM(n.valor_icms) AS icms_nfe
FROM dbo.nfe AS n
WHERE n.situacao = 'AUTORIZADA'
      AND n.tipo_operacao = '1'
GROUP BY n.id_emitente;
GO
```

O `WHERE` desta view responde: **quais documentos entram na agregacao?**

Passo 2 - Criar uma view para a EFD

Crie outra view, tambem com uma linha por contribuinte. Considere somente documentos regulares de saida:

```
CREATE OR ALTER VIEW dbo.vw_roteiro_efd_contribuinte AS
SELECT e.id_contribuinte,
       COUNT(*) AS qtd_efd,
       SUM(e.valor_documento) AS valor_efd,
       SUM(e.valor_icms) AS icms_efd
FROM dbo.efd_c100 AS e
WHERE e.ind_oper = '1'
      AND e.cod_situacao = '00'
GROUP BY e.id_contribuinte;
GO
```

Passo 3 - Consultar as views com juncoes

Junte as duas views ao cadastro de contribuintes. Use **LEFT JOIN** para manter no resultado contribuinte que tenha NF-e, EFD ou somente um dos dois:

```
SELECT c.id_contribuinte,
       c.razao_social,
       COALESCE(n.qtd_nfe, 0) AS qtd_nfe,
       COALESCE(n.valor_nfe, 0) AS valor_nfe,
       COALESCE(e.qtd_efd, 0) AS qtd_efd,
       COALESCE(e.valor_efd, 0) AS valor_efd,
       COALESCE(n.valor_nfe, 0) - COALESCE(e.valor_efd, 0) AS diferenca_valor
FROM dbo.contribuinte AS c
LEFT JOIN dbo.vw_roteiro_nfe_contribuinte AS n
      ON n.id_contribuinte = c.id_contribuinte
LEFT JOIN dbo.vw_roteiro_efd_contribuinte AS e
      ON e.id_contribuinte = c.id_contribuinte
WHERE c.regime_tributario = 'NORMAL'
ORDER BY c.razao_social;
```

Passo 4 - Aplicar **HAVING** sobre os grupos

Agora exiba somente contribuintes do regime **NORMAL** cujo total de NF-e autorizadas de saída seja superior a R\$ 100.000. Como a consulta final possui **GROUP BY**, o limite deve ser aplicado com **HAVING**:

```
SELECT c.id_contribuinte,
       c.razao_social,
       MAX(COALESCE(n.qtd_nfe, 0)) AS qtd_nfe,
       MAX(COALESCE(n.valor_nfe, 0)) AS valor_nfe,
       MAX(COALESCE(e.qtd_efd, 0)) AS qtd_efd,
       MAX(COALESCE(e.valor_efd, 0)) AS valor_efd,
       MAX(COALESCE(n.valor_nfe, 0))
       - MAX(COALESCE(e.valor_efd, 0)) AS diferenca_valor
FROM dbo.contribuinte AS c
LEFT JOIN dbo.vw_roteiro_nfe_contribuinte AS n
      ON n.id_contribuinte = c.id_contribuinte
LEFT JOIN dbo.vw_roteiro_efd_contribuinte AS e
      ON e.id_contribuinte = c.id_contribuinte
WHERE c.regime_tributario = 'NORMAL'
GROUP BY c.id_contribuinte, c.razao_social
HAVING MAX(COALESCE(n.valor_nfe, 0)) > 100000
ORDER BY valor_nfe DESC;
```

O **WHERE** seleciona contribuintes do regime **NORMAL**, antes do agrupamento. O **HAVING** seleciona grupos cujo valor agregado de NF-e supera R\$ 100.000.

Passo 5 - Comparar WHERE e HAVING

Para experimentar o filtro de linha, recrie temporariamente a view

`vw_roteiro_nfe_contribuinte` acrescentando `n.valor_total > 1000` ao `WHERE` da tabela `nfe`:

```
CREATE OR ALTER VIEW dbo.vw_roteiro_nfe_contribuinte AS
SELECT n.id_emitente AS id_contribuinte,
       COUNT(*) AS qtd_nfe,
       SUM(n.valor_total) AS valor_nfe,
       SUM(n.valor_icms) AS icms_nfe
FROM dbo.nfe AS n
WHERE n.situacao = 'AUTORIZADA'
      AND n.tipo_operacao = '1'
      AND n.valor_total > 1000
GROUP BY n.id_emitente;
GO
```

Depois de recriar a view com esse filtro, execute a consulta do Passo 3. Compare com uma consulta que mantém a view original, soma todas as notas autorizadas e usa `HAVING` para filtrar contribuintes:

```
HAVING MAX(COALESCE(n.valor_nfe, 0)) > 1000
```

Na primeira variacao, uma nota abaixo de R\$ 1.000 sai da conta. Na segunda, todas as notas entram na soma e somente o contribuinte com total acima do limite e mantido. Depois do experimento, recrie a view sem `n.valor_total > 1000` para restaurar a definicao original.

Conferencia da questao 2

Verificacao	Resposta correta
Contribuintes cadastrados	60
Contribuintes do regime <code>NORMAL</code>	executar a consulta
Linhas apos <code>HAVING valor_nfe > 100000</code>	executar a consulta
Resultado de <code>HAVING</code> com <code>valor_nfe > 100000</code> e <code>WHERE n.valor_total ></code>	diferente do anterior

1000

Soma de NF-e autorizadas de saida usada na conferencia	deve vir da viewvwroteiro_nfe_contribuinte
--	--

Soma de EFD regular de saida usada na conferencia	deve vir da viewvwroteiro_efd_contribuinte
---	--

Consulta de conferencia sugerida:

```
SELECT COUNT(*) AS grupos_encontrados,  
       SUM(COALESCE(n.qtd_nfe, 0)) AS documentos_nfe,  
       SUM(COALESCE(e.qtd_efd, 0)) AS documentos_efd  
FROM   dbo.contribuinte AS c  
LEFT JOIN dbo.vwroteiro_nfe_contribuinte AS n  
       ON n.id_contribuinte = c.id_contribuinte  
LEFT JOIN dbo.vwroteiro_efd_contribuinte AS e  
       ON e.id_contribuinte = c.id_contribuinte  
WHERE  c.regime_tributario = 'NORMAL'  
       AND COALESCE(n.valor_nfe, 0) > 100000;
```

Questao 3 - A mesma consulta com JOIN e subconsulta

Objetivo

Encontrar contribuintes que possuem pelo menos uma NF-e autorizada de saida que tambem aparece como documento regular na EFD. A questao deve ser feita sem **GROUP BY**, sem **COUNT**, sem **SUM** e sem qualquer agregacao.

Tabelas selecionadas

- **dbo.contribuinte AS c**
- **dbo.nfe AS n**
- **dbo.efd_c100 AS e**

Passo 1 - Solucao com juncoes

Escreva uma consulta que retorne apenas **id_contribuinte**, **cnpj** e **razao_social**.
Use **DISTINCT**, pois um contribuinte pode ter varias notas:

```
SELECT DISTINCT c.id_contribuinte, c.cnpj, c.razao_social
FROM dbo.contribuinte AS c
INNER JOIN dbo.nfe AS n
    ON n.id_emitente = c.id_contribuinte
    AND n.situacao = 'AUTORIZADA'
    AND n.tipo_operacao = '1'
INNER JOIN dbo.efd_c100 AS e
    ON e.id_contribuinte = c.id_contribuinte
    AND e.chave_acesso = n.chave_acesso
    AND e.ind_oper = '1'
    AND e.cod_situacao = '00';
```

Passo 2 - Reescrever com **EXISTS**

Substitua as juncoes com documentos por subconsultas correlacionadas. A consulta externa deve continuar retornando uma linha por contribuinte:

```
SELECT c.id_contribuinte, c.cnpj, c.razao_social
FROM dbo.contribuinte AS c
WHERE EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    INNER JOIN dbo.efd_c100 AS e
        ON e.chave_acesso = n.chave_acesso
        AND e.id_contribuinte = c.id_contribuinte
        AND e.ind_oper = '1'
        AND e.cod_situacao = '00'
    WHERE n.id_emitente = c.id_contribuinte
        AND n.situacao = 'AUTORIZADA'
        AND n.tipo_operacao = '1'
);
```

Passo 3 - Comparar as formas

1. Execute as duas consultas.
2. Compare a quantidade de linhas.
3. Confira se os conjuntos de **id_contribuinte** sao iguais.
4. Retire **DISTINCT** da primeira consulta e explique a duplicacao.
5. Explique por que **EXISTS** nao precisa de **DISTINCT**: ele responde apenas se encontrou pelo menos uma linha.

Conferencia da questao 3

Verificacao	Resposta correta
Linhas da consulta com JOIN e DISTINCT	igual a consulta com EXISTS
Linhas da consulta com EXISTS	executar a consulta
Diferenca entre os conjuntos de IDs	0
Uso de GROUP BY , SUM ou COUNT	nenhum
Consulta com JOIN sem DISTINCT	pode repetir contribuintes

Conferencia dos conjuntos:

```
-- A consulta abaixo deve retornar zero.
SELECT id_contribuinte FROM (
    -- coloque aqui o resultado da consulta com JOIN
) AS com_join
EXCEPT
SELECT id_contribuinte FROM (
    -- coloque aqui o resultado da consulta com EXISTS
) AS com_exists;
```

Questao 4 - Auditores acima da media de valor principal

Objetivo

Encontrar os auditores que realizaram OSs com autos de infração cujo valor principal acumulado ficou acima da media dos totais acumulados por auditor.

A atividade combina:

- juncoes;
- **GROUP BY** por auditor;
- **SUM(valor_principal)**;
- **HAVING**;
- subconsulta escalar para calcular a media.

A media deve considerar somente auditores que possuem pelo menos um auto. Auditores sem autos nao entram no denominador.

Tabelas selecionadas

- `dbo.auditor_fiscal AS af`
- `dbo.ordem_servico AS os`
- `dbo.auto_infracao AS ai`

Passo 1 - Calcular o total por auditor

Comece com a consulta sem o filtro da media:

```
SELECT af.matricula,
       af.nome,
       COUNT(DISTINCT os.id_os) AS qtd_os_com_auto,
       COUNT(DISTINCT ai.id_auto) AS qtd_autos,
       SUM(ai.valor_principal) AS valor_principal_total
FROM dbo.auditor_fiscal AS af
INNER JOIN dbo.ordem_servico AS os
         ON os.matricula_auditor = af.matricula
INNER JOIN dbo.auto_infracao AS ai
         ON ai.id_os = os.id_os
GROUP BY af.matricula, af.nome
ORDER BY valor_principal_total DESC;
```

Anote quantas linhas foram produzidas. Elas representam auditores com pelo menos um auto, nao necessariamente os 20 auditores cadastrados.

Passo 2 - Calcular a media em uma subconsulta

A subconsulta deve repetir a agregacao por auditor e devolver uma unica coluna de totais. Depois, use `AVG` sobre esses totais:

```
SELECT AVG(x.valor_principal_total) AS media_valor_principal_por_auditor
FROM (
    SELECT af2.matricula,
           SUM(ai2.valor_principal) AS valor_principal_total
    FROM dbo.auditor_fiscal AS af2
    INNER JOIN dbo.ordem_servico AS os2
```

```
        ON os2.matricula_auditor = af2.matricula
INNER JOIN dbo.auto_infracao AS ai2
        ON ai2.id_os = os2.id_os
GROUP BY af2.matricula
) AS x;
```

Passo 3 - Colocar a subconsulta no **HAVING**

Complete a consulta para que o **HAVING** compare o total de cada auditor com a media calculada pela subconsulta:

```
SELECT af.matricula,
       af.nome,
       COUNT(DISTINCT os.id_os) AS qtd_os_com_auto,
       COUNT(DISTINCT ai.id_auto) AS qtd_autos,
       SUM(ai.valor_principal) AS valor_principal_total
FROM   dbo.auditor_fiscal AS af
INNER JOIN dbo.ordem_servico AS os
        ON os.matricula_auditor = af.matricula
INNER JOIN dbo.auto_infracao AS ai
        ON ai.id_os = os.id_os
GROUP BY af.matricula, af.nome
HAVING SUM(ai.valor_principal) > (
    SELECT AVG(x.valor_principal_total)
    FROM (
        SELECT af2.matricula,
               SUM(ai2.valor_principal) AS valor_principal_total
        FROM   dbo.auditor_fiscal AS af2
        INNER JOIN dbo.ordem_servico AS os2
                ON os2.matricula_auditor = af2.matricula
        INNER JOIN dbo.auto_infracao AS ai2
                ON ai2.id_os = os2.id_os
        GROUP BY af2.matricula
    ) AS x
)
ORDER BY valor_principal_total DESC;
```

Passo 4 - Interpretar o resultado

Para cada auditor retornado, responda:

1. Quantas OSs com auto ele realizou?
2. Quantos autos foram lavrados nessas OSs?
3. Qual e o valor principal total?
4. Por que um auditor com varias OSs pode nao aparecer?

5. Por que a media nao deve ser calculada diretamente sobre as 72 linhas de autos?

Conferencia da questao 4

Verificacao	Resposta correta
Auditores cadastrados	20
OSs na base	150
Autos na base	72
Auditores com pelo menos um auto	resultado do Passo 1
Media dos totais por auditor	resultado do Passo 2
Auditores acima da media	resultado do Passo 3
Autos pertencentes aos auditores acima da media	executar conferencia

Consulta de conferencia unica:

```
WITH totais AS (  
    SELECT af.matricula, af.nome,  
           COUNT(DISTINCT os.id_os) AS qtd_os_com_auto,  
           COUNT(DISTINCT ai.id_auto) AS qtd_autos,  
           SUM(ai.valor_principal) AS valor_principal_total  
    FROM dbo.auditor_fiscal AS af  
    INNER JOIN dbo.ordem_servico AS os  
           ON os.matricula_auditor = af.matricula  
    INNER JOIN dbo.auto_infracao AS ai  
           ON ai.id_os = os.id_os  
    GROUP BY af.matricula, af.nome  
)  
), media AS (  
    SELECT AVG(valor_principal_total) AS media_valor_principal  
    FROM totais  
)  
SELECT COUNT(*) AS auditores_acima_media,  
       SUM(qtd_autos) AS autos_dos_auditores_acima_media,  
       MIN(media.media_valor_principal) AS media_usada  
FROM totais  
CROSS JOIN media  
WHERE totais.valor_principal_total > media.media_valor_principal;
```

A CTE da conferencia tem a mesma granularidade da consulta da atividade: uma linha por auditor com auto. Isso evita comparar um auditor com o valor de um

auto individual e evita que auditores com muitos autos tenham peso indevido na media.

Entrega esperada

Entregue um arquivo `lab_agregacao_juncoes.sql` contendo:

1. a criacao da `vw_roteiro_produtividade_auditor`;
2. as consultas dos quatro exercicios;
3. os resultados obtidos nas tabelas de conferencia;
4. respostas curtas para as perguntas de interpretacao;
5. uma observacao sobre a diferenca entre filtrar linhas (`WHERE`) e filtrar grupos (`HAVING`).

Critérios de avaliacao

Criterio	Peso
Juncoes corretas e chaves usadas	25%
Escolha entre <code>INNER JOIN</code> e <code>LEFT JOIN</code>	20%
Agregacao na granularidade correta	25%
Diferenciacao entre <code>WHERE</code> e <code>HAVING</code>	20%
Uso correto de subconsultas e interpretacao	10%